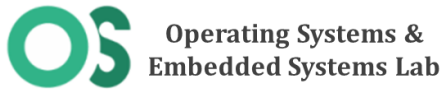
Operating Systems Lab 1
Shared Memory & Message Passing

Due date: 2026/10/16 17:00 (before lab1 course finishes)

oslab@mail.csie.ncku.edu.tw
TA: p76144906@gs.ncku.edu.tw、vx6142035@gs.ncku.edu.tw、p76151149@gs.ncku.edu.tw



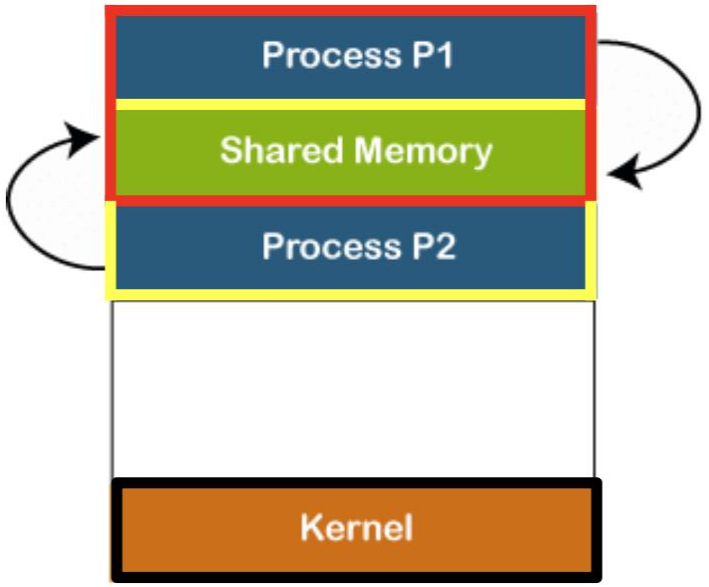


Outline

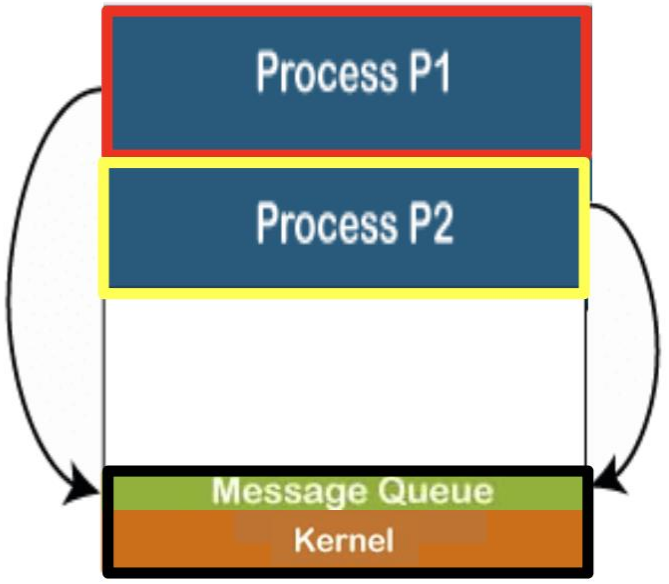
- Overview
- Requirement & Flow
- Related Works
- Demo & Grading
- Precautions

Overview - Inter Process Communication (IPC)

Approaches to Inter Process Communication

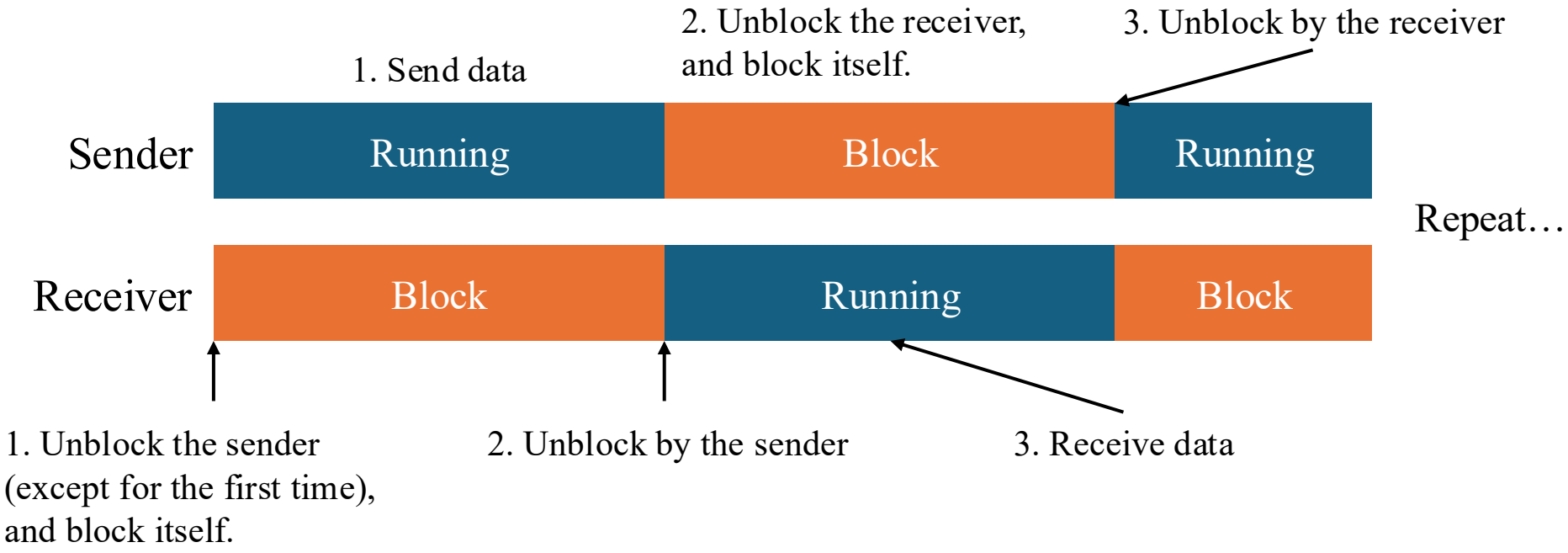


Shared Memory



Message Queue

Requirement & Flow



Requirement & Flow

Implement sender–receiver communication with two wrappers, both in one source file: **lab1.c**

1. send(message, &mailbox)
2. receive(&message, &mailbox)

main() function in lab1.c reads argv[1] (sender or receiver) and runs that role.

And implement these wrappers with two mechanisms

1. Message Passing (Using Message Passing system calls)
2. Shared Memory (Using Shared Memory)

Requirement & Flow

Sender

```
$ ./lab1 sender 1 ./message.txt
```

1. Call send(message, &mailbox) according to the flow in page 4.
2. Measure the total sending time.
3. Get the mechanism and the input file from command line arguments.
   (1 for Message Passing, 2 for Shared Memory)
1. Get the messages to be sent from the input file.
2. Print information on the console according to the output format.
3. If the message from the input file is EOF, send an exit message to the receiver.
4. Print the total sending time and terminate the sender.

Receiver

```
$ ./lab1 receiver 1
```

1. Call receive(&message, &mailbox) according to the flow in page 4.
2. Measure the total receiving time.
3. Get the mechanism from command line arguments.
4. Print information on the console according to the output format.
5. If the exit message is received, print the total receiving time and terminate the receiver.

Requirement & Flow

Lines of messages

- Message size: 1-1024 bytes.
- No blank lines.

```
first message
second message
third message
fourth message
fifth message
sixth message
seventh message
eighth message
ninth message
tenth message
eleventh message
twelfth message
thirteenth message
fourteenth message
fifteenth message
```

Requirement & Flow

Output example

Sender

```
wilicw@wilicw-ASUSPRO-D840MB-M840MB ~/os_2026_lab1_template (main)> ./lab1 sender 1 ./message.txt
Message passing
Sent: first message
Sent: second message
Sent: third message
Sent: fourth message
Sent: fifth message
Sent: sixth message
Sent: seventh message
Total time in sending message: 0.000061s
```

Receiver

```
wilicw@wilicw-ASUSPRO-D840MB-M840MB ~/os_2026_lab1_template (main)> ./lab1 receiver 1
Message passing
Received: first message
Received: second message
Received: third message
Received: fourth message
Received: fifth message
Received: sixth message
Received: seventh message
Received: Total time in receiving message: 0.000045s
```

Requirement & Flow

Time Measurement

Measure only actions related to communication.

- Sending / receiving messages via Message Passing system calls.
- Accessing the shared memory.

Don't measure actions unrelated to communication.

- Waiting to be unblocked.
- Printing messages.

Example:

```c
#include <time.h>
struct timespec start, end;
double time_taken;

clock_gettime(CLOCK_MONOTONIC, &start);
send(message, &mailbox);
clock_gettime(CLOCK_MONOTONIC, &end);

time_taken = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) * 1e-9;
```

Related Works — APIs

	System V API	**POSIX API**
Semaphore	semget(), semop(), semctl()	sem_open(), sem_wait(), sem_post(), sem_close(), sem_unlink()
Shared Memory	shmget(), shmat(), shmdt(), shmctl()	shm_open(), mmap(), munmap(), shm_unlink()
Message Passing	msgget(), msgsnd(), msgrcv(), msgctl()	mq_open(), mq_send(), mq_receive(), mq_close(), mq_unlink()

Related Works — Semaphore

- Semaphore can be used for Synchronization
- Semaphore S – integer variable
- Two standard operations modify S: wait() and signal()

```
wait(S) {
    while S<=0
        ; // no-op
    S--;
}


signal(S) {
    S++;
}
```

Related Works — Deadlock

Deadlock – two or more processes are waiting infinitely for an event that can be caused by only one of the waiting processes.

- Let S and Q be two semaphores initialized to 1.
- Consider if P_0 executes wait(S) and P_1 wait(Q). When P_0 executes wait(Q), it must wait for P_1. However, P_1 also waits for P_0 when it executes wait(S).

If the semaphore was (accidently) initialized as 0, it could also cause the deadlock because no process can get the semaphore.

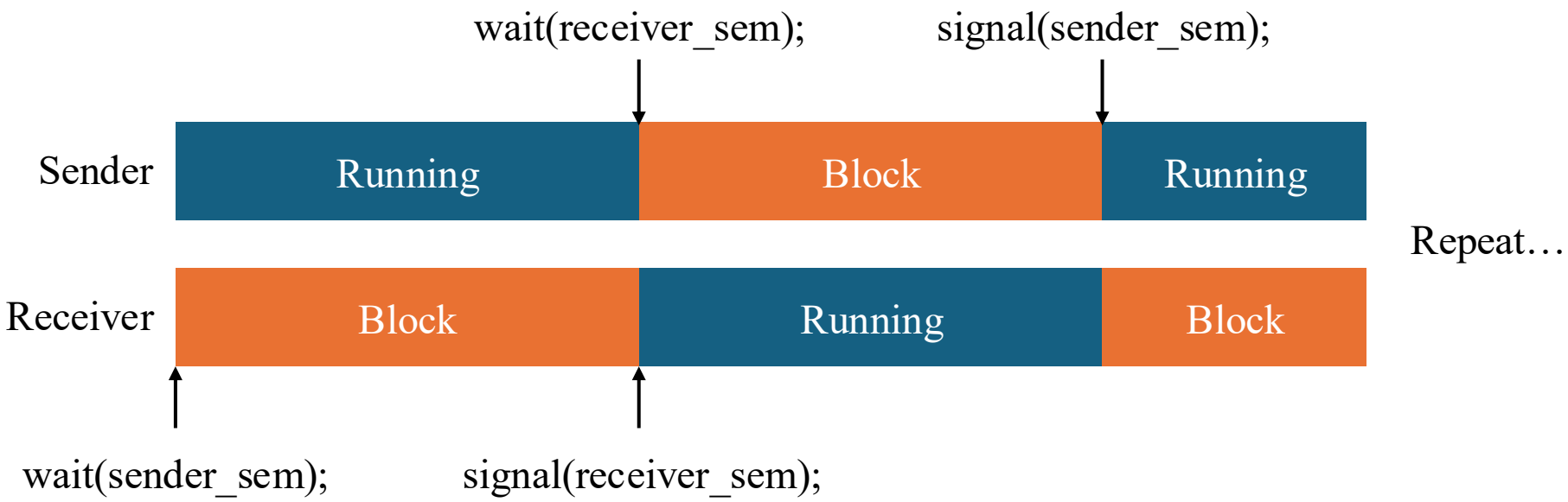
P_0

```
wait(S);
wait(Q);
   .
   .
   .
signal(S);
signal(Q);
```

P_1

```
wait(Q);
wait(S);
   .
   .
   .
signal(Q);
signal(S);
```

Related Works — Semaphore



Demo & Grading

- (2.5pt) Show communication information based on Message Passing. (follow the output format)

- (2.5pt) Show communication information based on Shared Memory. (follow the output format)

- (2pt) Compare their performance according to these communication information. And Shared-Memory shall be faster than Message-Passing.

- (3pt) Handwritten Quiz.

Precautions

Due Date: **2026/10/16 17:00** (before lab1 course finishes)

- You should implement lab1 with only **C** language.
- There are 4 essential files: lab1.c/.h, message.txt (input file) and Makefile.

And you can get it from os_2026_lab1_template.

- You can modify the makefile as you want, but make sure it can compile your codes and create the executable successfully.